



UNIVERSIDADE FEDERAL DO CEARÁ
Campus de Quixadá
Prof. Atilio Luiz
QXD01115- Estrutura de Dados Avançada

AP1
2024.1

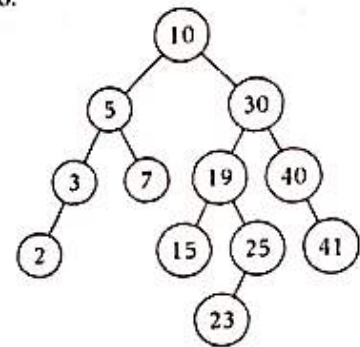
Nome: _____

Matrícula: _____

- [2 pontos] 1. Seja T uma árvore AVL inicialmente vazia. Mostre cada uma das árvores AVL intermediárias que resultam após a inserção bem-sucedida das chaves 7, 8, 10, 16, 14, 12, 5, 3, 4, 9, 2 (nesta ordem) na árvore T . Para cada vez que uma operação de rotação acontecer, escreva qual rotação está sendo executada e qual o nó pivô da rotação.

- [2 pontos] 2. Dada a árvore AVL da figura ao lado, responda os dois itens abaixo:

- (a) Desenhe a árvore binária de busca obtida logo após o momento em que o nó 7 é removido, mas antes que qualquer rebalanceamento aconteça. No mesmo desenho, rotule cada nó da árvore resultante com o valor do seu balanço.
- (b) Rebalanceie a árvore que resultou do item (a), a fim de torná-la AVL. Desenhe uma nova árvore para cada rotação que ocorrer e escreva qual rotação está sendo feita.



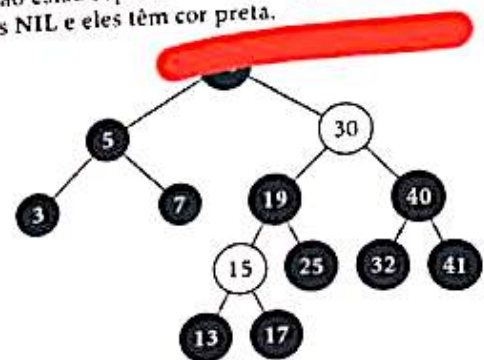
- [12 pontos] 3. Desenhe cada uma das árvores rubro-negras que resultam após a inserção bem-sucedida das chaves 80, 51, 38, 21, 25, 10 (nesta ordem) em uma árvore rubro-negra T inicialmente vazia. Para cada vez que uma inserção ocasionar um desregulamento da árvore, indique qual dos 3 casos de regulamento visto em aula está sendo aplicado (ou seja, escreva o caso e as operações de recoloração de nós e de rotação (se houver)).

Atenção: Siga o algoritmo que foi ensinado em sala. Algoritmos inventados da sua cabeça não serão aceitos.

- [2 pontos] 4. Considere a árvore rubro-negra T na figura abaixo, onde os nós pretos estão pintados de cor preta e os nós vermelhos são os nós em cor branca. Os nós NIL não estão representados na figura, mesmo assim considere que cada nó folha nessa figura tem dois filhos NIL e eles têm cor preta.

- (a) Rebalanceie a árvore que resulta após a remoção do nó com chave 7. Desenhe uma nova árvore para cada operação de rebalanceamento que você fizer e escreva explicitamente quais as operações que estão sendo feitas. O rebalanceamento deve ser feito de acordo com o algoritmo ensinado em sala de aula. Lembre-se que o algoritmo de rebalanceamento da remoção na árvore rubro-negra possui 4 casos.

Atenção: Algoritmos inventados da sua cabeça não serão aceitos.



Nota: _____

5. Uma fila de prioridades é uma estrutura de dados que mantém uma coleção A de elementos na qual cada um de seus elementos possui uma *prioridade*, isto é, um valor numérico. As listas de prioridades existem em duas formas: listas de prioridades mínimas e listas de prioridades máximas. Em sala de aula, vimos como implementar (de forma teórica e prática) uma fila de prioridade máxima, de forma eficiente, usando como base uma outra estrutura chamada *heap de máximo*. Nesta questão, a estrutura de dados `MaxPriorityQueue` é uma fila de prioridade de máximo, onde os seus elementos são números inteiros (as prioridades). Essa estrutura de dados possui basicamente as 7 funções públicas listadas a seguir:

- o construtor `MaxPriorityQueue()`, que cria uma fila de prioridades vazia em tempo $O(1)$.
- o construtor `MaxPriorityQueue(CONST VECTOR<INT>& VEC)`, que recebe como parâmetro o vetor `VEC` e cria uma fila de prioridades em tempo $O(n)$, contendo todos os elementos do vetor `VEC`, onde $n = \text{VEC.SIZE}()$.
- a função `TOP()` que retorna o valor do elemento de maior prioridade (mas não o remove da fila). Essa função tem complexidade $O(1)$.
- a função `POP()` que retorna o valor do elemento de maior prioridade e o remove da fila. Essa função executa em tempo $O(\lg n)$.
- a função `PUSH(INT v)` que recebe um inteiro v e o insere na fila em tempo $O(\lg n)$. Essa função não retorna nada.
- a função `EMPTY()` que retorna o valor booleano `TRUE` se, e somente se, a fila estiver vazia. Essa função tem complexidade $O(1)$.
- a função `SIZE()` que retorna um valor inteiro, que é o número de elementos atualmente na fila. Essa função tem complexidade $O(1)$.

Com base nos conhecimentos adquiridos com filas de prioridades, resolva os problemas a seguir.

(a) [2 pontos] Usando a estrutura de dados `MaxPriorityQueue` e suas funções públicas, escreva em C++ um algoritmo que ordena um vetor de inteiros A com n inteiros, em tempo $O(n \lg n)$. Você deve escrever uma função chamada `ORDENA_VETOR(INT A[], INT N)` que recebe como entrada o vetor A com n inteiros e o seu tamanho n e o ordena. O retorno dessa função é do tipo `void`. Dentro dessa função, ela deve fazer uso da estrutura de dados `MaxPriorityQueue` definida acima. Ao final, argumente porque a complexidade da sua função é $O(n \lg n)$.

Atenção: NÃO estou pedindo para você implementar as funções da `MaxPriorityQueue`. Se você fizer isso, você não ganhará nenhum ponto pela questão. Estou permitindo você usar a estrutura de dados, ela já está pronta para uso. Então, use a estrutura de dados `MaxPriorityQueue` a fim de resolver o problema que foi pedido.

b) [Bônus (1 ponto)] Dado um array A de n inteiros distintos (os elementos podem estar completamente desordenados, não sabemos a procedência) e um número k , onde k é menor que o tamanho do array, escreva em C++ um algoritmo que encontra o k -ésimo menor elemento do vetor A , em tempo $O(n \lg k)$.

Você deve escrever uma função chamada `PEGA_O_K_ESIMO_MENOR(INT A[], INT N)` cujo tipo de retorno é `int`. Ao final, argumente porque a sua função tem complexidade $O(n \lg k)$.